

Programming Assignments #4 and 5

CS 302 Programming Methodologies and Software Imp.

REQUIREMENTS DOCUMENT

The Overall Approach:

With the last two programming assignments, you will be implementing in Python using VS Code. Your goal must be to develop a structured and hierarchical solution using object-oriented programming, data structures and recursion in Python. You will find that this assignment is structured differently than previous assignments. The progress submissions comprise a significant number of points and should not be missed. Please read this requirements document carefully.

This assignment is structured into two programs:

- Program #4 will focus on a specification that you will be writing, the core hierarchy of the system and a comprehensive PyTest black-box test suite. The core hierarchy will have a base class with three derived types which are similar but different.
- There needs to be at least two instance variables (e.g., data members) for each class and at least three methods that represent the job of each class. It is expected that all data will be treated as protected (or private if necessary).
- It is also expected that variable names used throughout will be self-documenting (no x, y, t, v etc. variables!).
- Program #5 will expand upon this foundation, incorporating data structures such as a balanced tree (2-3 tree or a b-tree) or a Binary Search Tree (BST). It is expected with Program #5 that a complete application will be built. You are to implement the tree which was not implemented for Program #3.

If you did not implement a balanced tree in Program #3, it will be important to implement a balanced tree in Python for Program #5. If you didn't implement a Binary Search Tree in Program #3, then it is time to implement one for program #5!

Program #4 Portfolio Objectives to be met:

1. Develop a comprehensive specification for the functionality of the core hierarchy.
2. Develop a comprehensive black box test suite using PyTest for the core hierarchy.
3. Implement a complete hierarchical solution using Python with proper implementation of the `super()`
4. Implement at least three methods per class with 2 instance variables per class in Python. Data must all be treated as “protected” or “private”
5. Experience constructors with arguments and exception handling in Python
6. Achieve readable, reuseable, and maintainable code

Program #5 Portfolio Objectives to be met:

1. Implement a complete application with a menu based interface representing the application
2. Implement at least three methods in support of **operator overloading** in Python
3. In the application, continue to properly and fully support exception handling.
4. Achieve readable, reuseable, and maintainable code
5. Program #5 - Provide a comprehensive implementation of a tree using recursive algorithms exclusively (no iteration). (As mentioned on Page 1)

Statement of the Problem Requirements:

Do you purchase items through Amazon or get your groceries delivered to you? It is definitely an easier way to quickly look for a specialty item and get it delivered quickly, but it is often the case that you have to weed through many options that are not right for you especially if you are looking to buy locally. You have decided to start up your own business to provide local deliver services for three types of items. The three type of items need to be similar (so you can push up the common elements) but yet different (to have their own functionality). For example, you may want to offer locally grown produce where the consumer gets to be part of the decision making as to which farm to support. Or maybe it is athletic apparel from Columbia or Nike. Another choice would be to offer food from local restaurants or food carts (like grub-hub) and the customer gets to select which restaurant and menu item they want to order. You get to pick the three types of delivery services.

The first step is to writeup a specification of what the application should do – specifically focusing on the core hierarchy.

GENERAL CODING REQUIREMENTS:

Your Python programs must follow these rules:

- **Object-Oriented Design:**
 - Use single inheritance with proper application of `super()`; there must be a minimum of 5 classes with 3 of them in a hierarchy.
 - The application that USES the hierarchy must be in a class on its own. It could be part of a hierarchy, so think about that. Main itself should be minimal.
 - Implement at least one constructor with arguments.
- **Encapsulation:**
 - All instance variables should be designated as private or protected using underscore(s).
 - As a general rule, avoid accessing the instance variables from outside of the class with the exception of your test suite.
 - This means your node must be a class with private or protected data; the node class will need `setleft`, `setright`, `getleft` and `getright` methods to access the corresponding references.
- **Recursion:**
 - All repetitive data structure algorithms are to be implemented recursively. This will give you the ability to apply recursion in a language without pass by reference.
- **Validation:**
 - Make sure to validate all input. Empty input is not allowed.
 - Properly use type conversion and exception handling with all input
- **Data Structures:**
 - Yes, you SHOULD use strings and Python lists.
 - Make sure to use a Python list at least once in your assignment **and a NumPy array** as well (refer to lab).
- **Operator Overloading:**
 - Implement at least two “operator” functions and experiment with how they work in Python (refer back to our lab on operator overloading).
- **Exception Handling:**
 - We expect a thorough implementation of **exception handling!**
- **You may NOT, under any circumstances, use code that is not your own such as from StackOverflow, ChatCPT, or Chegg, or any such website which you can claim as your own.**

Summary of Deliverables for Programs 4 and 5:

1. **First Submission – Specification (instead of a .h file)**
 - a. **DRAFT Specification and UML diagram is due**
 - b. The first thing to submit is a draft of your specification so we can get an idea of the application that you are intending to build.
2. **First Progress Submission – Pytest Test Suite**
 - a. **Turn in a test plan and PyTest Test Suite**
 - b. Based on that specification, create a draft PyTest test suite representing “black box testing”.
 - c. Turn in an updated specification with your test suite
3. **Program #4 Due – Core Hierarchy with updated Spec and PyTest test suite**
 - a. **Program #4 represents the core hierarchy**
 - b. This includes **the core hierarchy, your completed Specification,** and updated **PyTest test** suite now representing “glass box testing”.
 - c. No other writeups required.
4. **Program #5 Due**
 - a. **Completed Program #5** with the data structures completed.
 - b. Include the revised Specification and Completed PyTest test suite.
 - c. Two writeups are also required (the efficiency writeup and an IDE writeup on how VS Code was used for debugging).

Step 1 – Develop a Specification

Treat this programming assignment document as a “requirements documentation”. Your job to determine what the user interface will be like and what three operations your derived classes will have. This is important to do before you start to code!

- **Your specification should be at least 600 words.**
- **It must include** what three types of products to deliver you will be supporting and their exact functionality (including arguments and returned information)
- **The specification must** include input and output formats (including prompts, saved states, and input expected.
- **Include error handling and validation rules.**
- The specification is required as your first deliverable for Program #4 submission. You will resubmit it at the end of Program #5 as well.
- Remember your name must be in everything submitted for us to grade!

Step 2 – Develop the Black Box Tests

Once the specification has been developed, create test plan and black box tests with PyTest based solely on the specification. If it isn't clear what you should test, then it means your specification isn't detailed enough! We expect that you will be resubmitting your specification with your PyTest suite as changes are likely to be needed.

Create a test module to be used with PyTest:

- Use your detailed specification to develop a test suite using “black box testing”. This is due at the end of the first week of the program's cycle.
- Then, update your test suite as each method is implemented, using “glass box testing”. With this, it should be clear to us that you have performed unit testing with every path through the software being examined! You will be turning this in with your finished core hierarchy (Program #4) and then again with any modifications when you turn in Program #5.

Final Step – Writeups to Include with the Completed Program #5

- Your final 400 word writeup should summarize what you have experienced with this assignment. I suggest keeping notes as you develop the test suite from your specification so that you can easily talk about what worked and what needed more detail. Did you create a test plan? Was it helpful? Did each and every path through your software get tested? What type of integration testing did you do?
- Instead of writing 200 words about a debugger, write about how you used VS Code to debug.

NEXT PAGE

*****The Specifics!*****

Program #4 – Writing the Specification

To make this feasible within a 2 ½ week cycle, we have just a few requirements which need to be planned – in detail – in your specification:

1. Decide upon the theme of your delivery service
 - a. Decide on your audience
 - b. What type of products do you want users to be able to purchase
 - c. Based on your decisions, this should help with deciding what type of user interface your software will have and what three types of products derived from a base class.
 - d. There should be a menu – plan it! Who is it aimed at? What choices will there be?
2. Decide upon the core hierarchy
 - a. Decide on the three types of products you want to support.
 - b. There must be two data (fields, instance variables) common among the three products (at least two fields which are pushed up to the base class) and data unique (at least two which stays derived).
 - c. Each class needs three different types of operations or functions – to figure out what these need to be, think about what someone would need to know to purchase that particular type of product. Since there is limited time, you will benefit by making good decisions early on and really focusing on being clear about how this program should work.
3. Decide what Operators you will support
 - a. Think about how operators apply? Would the + operator make sense to add a product to a bag (the data structure)? (program 5)
4. Planning the Application (Program 5)
 - a. Think about how the application will work and what choices there will be

The key with this hierarchy will be to plan what functions **and operators** make the most sense. This will require building a specification for this hierarchy first and then building a test suite for just these classes!

Program #4 – Building the Black Box test suite and Core Hierarchy

Once you have made decisions about what users, products, classes, and methods you will have, then build your Test Plan and Black Box PyTest test suite for testing each of those functions.

The common elements of the three types of products should be pushed up and the differences derived. Do not reimplement concepts that another class already implements. There needs to be at least THREE methods, besides constructors,

destructors, and display methods. If one of these classes needs a list of items – use built in options such as a Python List or NumPy's array.

*****PROGRAM #4's Deliverable** is the CORE HIERARCHY completed with a fully functional test suite using PyTest

Program #5 – Working with Collections of Data and the Application

Now it is time to manage a collection of products using data structures. In Program #5, put this together in the application. ***You **SHOULD** modify the original specification to now include the application and resubmit with the finished assignment***

You will be required to implement a tree to manage the bag of products. These should be mixed together all in the same data structure. Remember:

- If a balanced tree was not implemented in Program #3, then implement a balanced tree now (either a red-black tree or a 2-3-4 tree is expected). Otherwise implement a binary search tree.
 - You are ***not*** expected to remove from a balanced tree.
 - Recursion is required
- For A BST implement a comprehensive set of functions (insert, retrieve, display, and remove an item) and implement these recursively.